

Task 4: Optimal Diffusion Step Selection for Latency-Sensitive Inference

1. Introduction

Iterative diffusion requires a trade-off between semantic quality and computational latency. This task identifies the "knee" in the performance curve for the Encoder-Decoder variant, ensuring that the final deployment maximizes utility per GFLOP.

2. Methodology

We evaluate the model across a wide range of step counts: $T \in \{4, 8, 16, 32, 64\}$. Each configuration is benchmarked for throughput (tokens/sec) and semantic accuracy (BERTScore) to identify the optimal equilibrium state.

👉 **Implementation:** [analysis/step_ablation.py](#)

3. Experimental Results (T4 Benchmark)

For the Encoder-Decoder model, the step count of **T=4** was selected as the production default. This decision was driven by the observation that increasing steps to T=16 offered substantial latency costs with diminishing semantic gains on the target Sanskrit paraphrasing task.

3.1 Subtask Results Table (Optimal Model - T4)

Following the standardized step ablation framework, the benchmarks for the T4 Encoder-Decoder configuration are summarized below:

Ablation Subtask	Metric / Status	T4 Model Result
1. Baseline Throughput ($T=64$)	Compute latency of deep diffusion chain	~5,600 ms (Severely throttled)
2. Optimized Latency ($T=4$)	Measure time with minimum stable steps	~ 340 ms (Production Target)
3. Semantic Stability Check	BERTScore delta across T values	~ 0.05 (Minimal gain with higher T)
4. Token Diversity Ratio	Uniqueness of output tokens at T4	0.82 (Consistent with baseline)

Ablation Subtask	Metric / Status	T4 Model Result
5. Computational Gain	Relative reduction in GFLOPs	~20x Reduction (vs T=64)
6. Output Integrity Test	Manual audit for grammatical failure	✅ Status: Valid structural outputs.

4. Analysis & Conclusion

The T4 model strikes the most effective balance for the 112M parameter Encoder-Decoder architecture. By truncating the diffusion chain to four steps, we reduce the computational footprint by over 90% compared to typical baseline schedules, making the system deployable on standard CPU/MPS hardware without sacrificing the structural essence of the generated Sanskrit slokas.